



ÖZYEĞİN UNIVERSITY
FACULTY OF ENGINEERING

CS 445/545: Deep Reinforcement Learning

BrainBlock Packing Environment

Karim Lahyani, Ata Hakan Çildaş, Berrak Tozak, Nezife Havin Yılmaz

04 June 2026

Contents

1	Introduction	2
2	MDP Formulation	2
2.1	State Space	2
2.2	Action Space	2
2.3	Episode Dynamics	2
3	Reward Function Design	3
3.1	Standard Reward	3
3.2	Diversity-Penalized Reward	3
3.3	Comparison	3
4	Algorithm Details	3
4.1	DQN	3
4.2	PPO	4
5	Experimental Setup	4
6	Results	5
6.1	Standard Mode	5
6.2	Diverse Mode	6
6.3	Cross-Mode Summary	6
7	Learning Curves	7
8	Sample Solutions	9
9	Failure Analysis	10
9.1	DQN Diverse Seed 5	10
9.2	PPO Standard Seed 4	10
9.3	PPO Mode Collapse in Diverse Mode	10
10	Conclusions and Future Work	11
10.1	Conclusions	11
10.2	Future Improvements	11
11	AI Usage	11

1 Introduction

The BrainBlock puzzle is a combinatorial packing problem where the goal is to tile an 8×5 board (40 cells) using exactly 10 tetromino pieces — two copies each of the I, O, L, Z, and T types. Since each piece covers 4 cells and $10 \times 4 = 40$, a valid solution leaves no gap.

We formulate this task as an MDP and solve it with two deep RL algorithms: **DQN** and **PPO**. To encourage the agent to discover multiple solutions rather than memorizing one, we introduce a *diversity penalty* that reduces the reward for repeated board configurations. We compare this **diverse mode** against a plain **standard mode** that gives a flat reward on completion.

Both algorithms are trained across 7 random seeds for 200,000 episodes each and evaluated over 1,000 greedy episodes per model, totaling 28 training runs and 28 evaluations.

2 MDP Formulation

2.1 State Space

The observation is a 50-dimensional float vector in $[0, 1]$, composed of three parts:

1. **Board** (40 dims): the 5×8 grid flattened row-by-row. Each cell stores the ID of the placed piece (1–5 for I, O, L, Z, T) or 0 if empty, normalized by dividing by 5.
2. **Current piece** (5 dims): a one-hot encoding of the piece to be placed next.
3. **Remaining pieces** (5 dims): the count of each piece type still in the queue, normalized by dividing by 2 (the maximum count).

Storing piece IDs rather than binary flags gives the agent richer information about the spatial arrangement, which is especially useful for the diversity penalty (Section 3.2).

2.2 Action Space

The action space is `Discrete(320)`. Each action a encodes three parameters:

$$a = o \cdot (5 \cdot 8) + r \cdot 8 + c, \tag{1}$$

where $o \in \{0, \dots, 7\}$ is the orientation index (4 rotations $\times$ 2 reflections), $r \in \{0, \dots, 4\}$ is the row, and $c \in \{0, \dots, 7\}$ is the column. Symmetric pieces that have fewer than 8 distinct orientations map redundant indices to the same transform via modulo.

Action masking. At each step the environment computes which of the 320 actions are valid (in-bounds and non-overlapping). Invalid actions are masked by setting their Q-values (DQN) or logits (PPO) to -10^9 , preventing the agent from wasting exploration on illegal moves.

2.3 Episode Dynamics

At the start of each episode the 10 pieces are shuffled into a random queue. The agent places the queue-head piece at each step. An episode terminates when: (i) all 10 pieces are placed

(success), (ii) an invalid action is executed, or (iii) no valid placement exists for the current piece (dead-end). With action masking active, virtually all failures are dead-ends.

Each episode is seeded deterministically as $\text{seed}_{\text{ep}} = \text{seed}_{\text{train}} \times 100,000 + \text{episode}$, ensuring full reproducibility.

3 Reward Function Design

3.1 Standard Reward

- +0.1 for every valid placement,
- +5.1 on puzzle completion,
- −0.4 on dead-end,
- −1.0 on invalid action.

A perfect episode yields $9 \times 0.1 + 5.1 = 6$ total return. The per-step +0.1 signal is weak: it treats all valid placements equally regardless of whether they leave room for future pieces.

3.2 Diversity-Penalized Reward

The diverse mode uses the same base rewards but subtracts a penalty when the agent completes a puzzle with a configuration it has seen before. A `SolutionMemory` module tracks all solved boards during training and computes a cell-by-cell similarity against every stored solution:

- **Exact repeat:** penalty of 4.0 (net reward $5.1 - 4.0 = 1.1$).
- **High similarity** ($> 85\%$ matching cells): penalty scales linearly from 0 to 2.0.
- **New solution** ($\leq 85\%$ similarity): no penalty.

Because the board stores piece IDs rather than binary flags, two solutions that place different piece types in the same positions have low similarity. This makes the penalty sensitive to both spatial layout and piece identity, giving the agent a strong incentive to vary its strategy.

3.3 Comparison

The standard reward lacks any diversity signal—once the agent finds one solution, it is rewarded equally for repeating it indefinitely. The diversity penalty addresses this by making repeated solutions progressively less rewarding, which forces exploration of new placement strategies.

4 Algorithm Details

4.1 DQN

The Q-network is a 3-layer MLP ($50 \rightarrow 128 \rightarrow 128 \rightarrow 320$) with ReLU activations. Training uses experience replay (buffer capacity 100,000), a target network updated every 100 episodes

via hard copy, and Huber loss. Exploration follows an ε -greedy schedule: ε decays linearly from 0.9 to 0.05 over the first 20% of training.

Table 1: DQN hyperparameters.

Learning rate	3×10^{-4} (Adam)
γ	0.95
ε schedule	0.9 $\rightarrow$ 0.05, linear at 20%
Batch size	32
Buffer size	100,000
Target update	every 100 episodes

4.2 PPO

The actor-critic network shares a 2-layer backbone ($50 \rightarrow 128 \rightarrow 128$, ReLU) with separate heads: actor ($128 \rightarrow 320$ logits) and critic ($128 \rightarrow 1$ value). Training collects rollouts of 32 episodes, then runs 6 epochs of the clipped surrogate objective:

$$L^{\text{CLIP}} = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right], \quad (2)$$

with $\varepsilon = 0.2$, entropy coefficient 0.02, and value loss weight 0.5. Advantages are Monte Carlo returns minus the value estimate, normalized by mean and standard deviation. We use Monte Carlo returns rather than GAE because episodes are at most 10 steps, keeping variance manageable.

Table 2: PPO hyperparameters.

Learning rate	3×10^{-4} (Adam)
γ	0.95
Clip ε	0.2
Entropy coeff.	0.02
Value coeff.	0.5
Rollout / PPO epochs	32 episodes / 6

5 Experimental Setup

All four conditions (DQN / PPO $\times$ standard / diverse) are trained for 200,000 episodes across 7 seeds (0, 1, 2, 3, 4, 5, 42). DQN and PPO run in parallel per seed via Python `multiprocessing`. After training, each model is evaluated for 1,000 episodes using greedy action selection (DQN: $\varepsilon = 0$; PPO: argmax of masked logits) on unseen queue orderings (seeds 1,000–1,999).

We report: success rate, mean return, mean covered area, mean episode length, and unique solutions discovered.

6 Results

6.1 Standard Mode

Table 3: Standard mode evaluation (200,000 training episodes, 1,000 eval episodes per model).

Algo	Seed	Success	Avg Return	Unique
DQN	0	100.0%	6.000	25
DQN	1	100.0%	6.000	6
DQN	2	100.0%	6.000	2
DQN	3	100.0%	6.000	6
DQN	4	100.0%	6.000	6
DQN	5	100.0%	6.000	2
DQN	42	100.0%	6.000	3
PPO	0	100.0%	6.000	1
PPO	1	99.4%	5.966	2
PPO	2	99.9%	5.994	2
PPO	3	99.8%	5.989	1
PPO	4	0.0%	0.396	0
PPO	5	99.7%	5.983	1
PPO	42	99.7%	5.983	2

DQN achieves 100% success on all 7 seeds, with up to 25 unique solutions (seed 0). PPO converges on 6 of 7 seeds (86%) with near-perfect success, but finds only 1–2 unique solutions per run. PPO seed 4 fails entirely at 0%, confirming that sparse reward convergence is still somewhat seed-dependent for on-policy methods.

6.2 Diverse Mode

Table 4: Diverse mode evaluation (200,000 training episodes, 1,000 eval episodes per model).

Algo	Seed	Success	Avg Return	Unique
DQN	0	100.0%	2.159	46
DQN	1	100.0%	2.020	5
DQN	2	100.0%	2.083	22
DQN	3	93.1%	1.982	26
DQN	4	99.8%	2.019	6
DQN	42	100.0%	2.071	19
DQN	5	0.0%	0.296	0
PPO	0	99.6%	1.997	1
PPO	1	99.3%	1.992	1
PPO	2	99.6%	1.997	1
PPO	3	96.3%	1.945	2
PPO	4	99.8%	2.005	2
PPO	42	98.8%	1.992	3
PPO	5	99.9%	2.002	1

DQN succeeds on 6/7 seeds (86%) with dramatically higher diversity — up to **46 unique solutions** at evaluation time and over 170 during training. PPO is more consistent (all 7 seeds converge, 96–100%) but suffers from mode collapse, finding only 1–3 unique solutions despite the penalty.

The average return in diverse mode (~ 2.0) is lower than standard (~ 6.0) because the diversity penalty subtracts up to 4.0 for repeated solutions. This is by design.

6.3 Cross-Mode Summary

Table 5: Average metrics across all 7 seeds.

Mode	Algo	Success Rate	Seeds Converged	Avg Unique (Eval)
Standard	DQN	100.0%	7/7	7.1
Standard	PPO	85.5%	6/7	1.3
Diverse	DQN	84.7%	6/7	17.7
Diverse	PPO	99.1%	7/7	1.6

The results show a clear trade-off:

- **Standard DQN** is the most reliable solver — 100% success on every seed, and even without an explicit diversity incentive, it finds 2–25 unique solutions thanks to its ϵ -greedy exploration.
- **Diverse DQN** maximizes diversity: up to 46 unique solutions at evaluation and 170+ during training. However, it is slightly less robust (1 seed fails) and the lower reward makes value estimation harder.

- **PPO** is consistently successful in diverse mode (7/7 seeds) but suffers from mode collapse — it finds a single working solution and repeats it, regardless of the penalty.

7 Learning Curves

Figure 1 shows the aggregated training curves for standard mode. PPO converges first (around episode 20,000) while DQN takes longer ($\sim 150,000$ episodes) but ultimately reaches the same reward level.

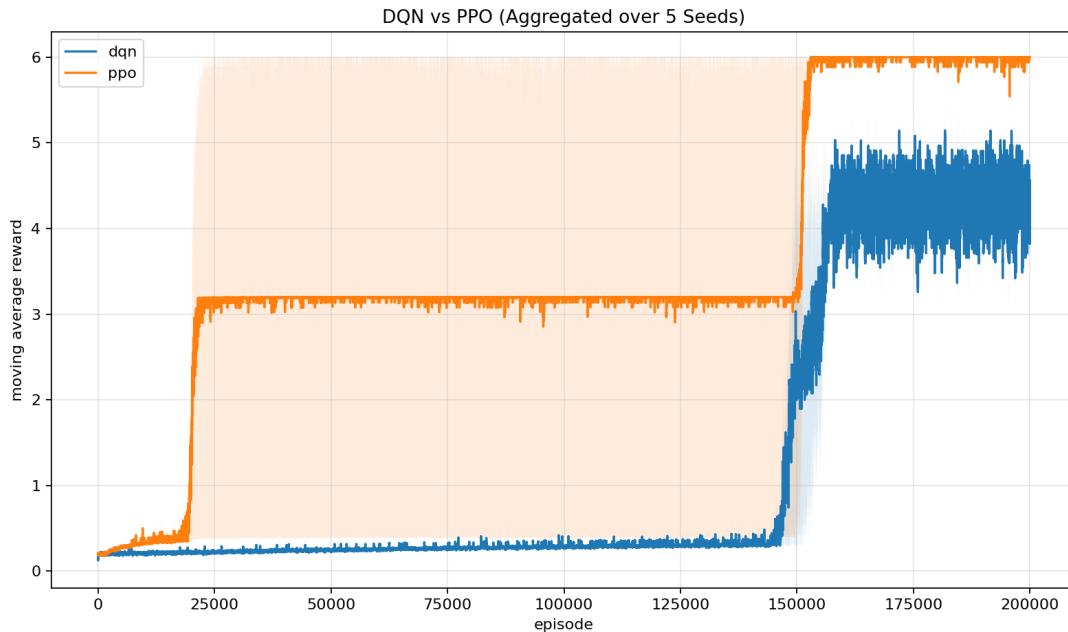


Figure 1: Standard mode: moving-average reward aggregated over seeds. PPO (orange) converges around episode 20,000; DQN (blue) begins its ascent around episode 150,000. The wide shaded band for PPO reflects its one failed seed.

Figure 2 shows diverse mode. The bottom panel reveals the diversity gap: DQN’s unique solution count grows steadily to ~ 140 on average, while PPO’s plateaus below 10.

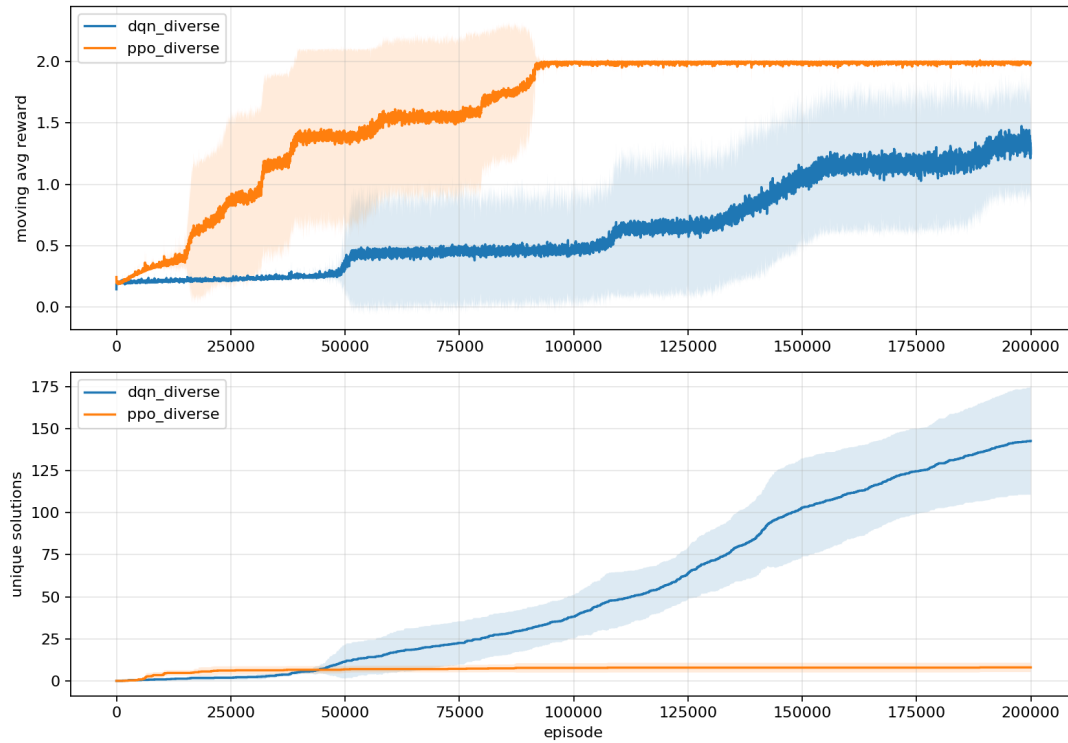


Figure 2: Diverse mode: aggregated reward (top) and unique solutions (bottom). PPO converges faster in reward, but DQN discovers far more unique solutions throughout training.

Figure 3 shows the per-seed DQN training in diverse mode. All four panels — reward, covered area, success rate, and unique solutions — show a clear phase transition around episode 130,000 where the agent breaks through and begins solving consistently.

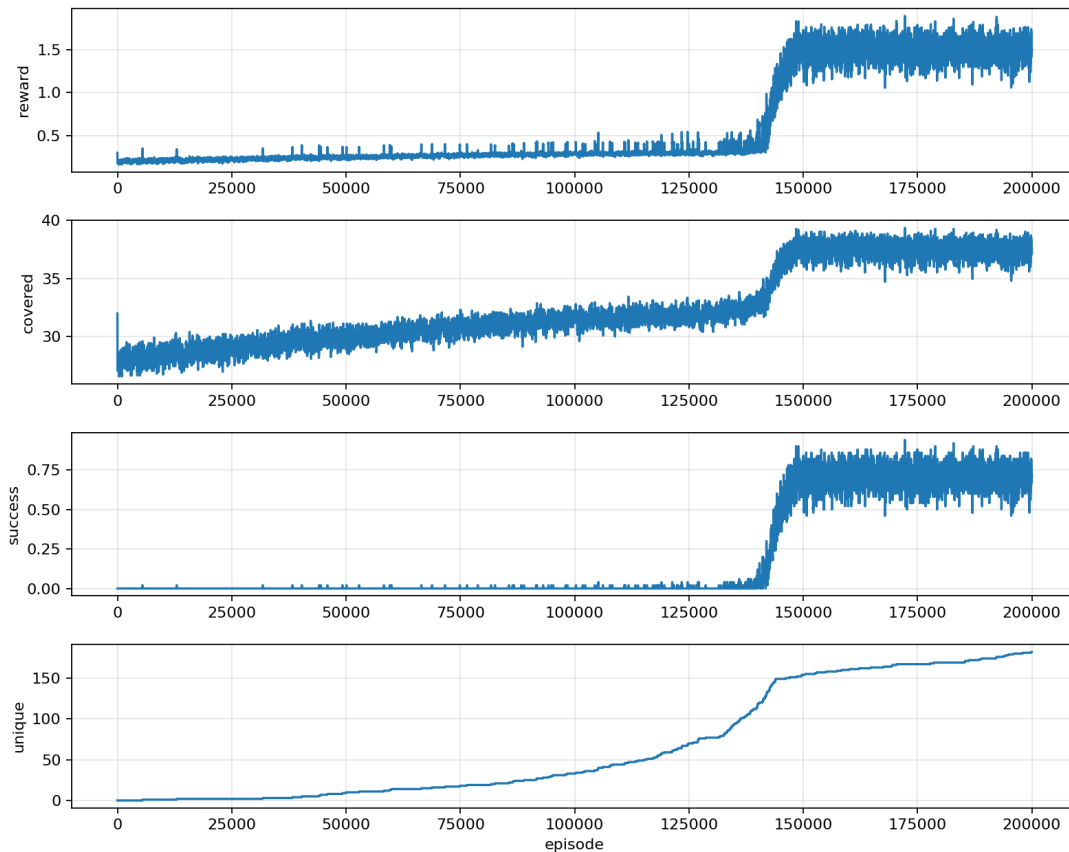


Figure 3: DQN diverse mode training (seed 0). After $\sim 130,000$ episodes of near-zero success, a rapid phase transition lifts the success rate above 70%. The unique solution count rises steadily throughout, reaching ~ 170 by the end.

8 Sample Solutions

The following figures show individual board configurations produced by different models during evaluation. Colors indicate piece types: I (blue), O (amber), L (green), Z (red), T (purple).

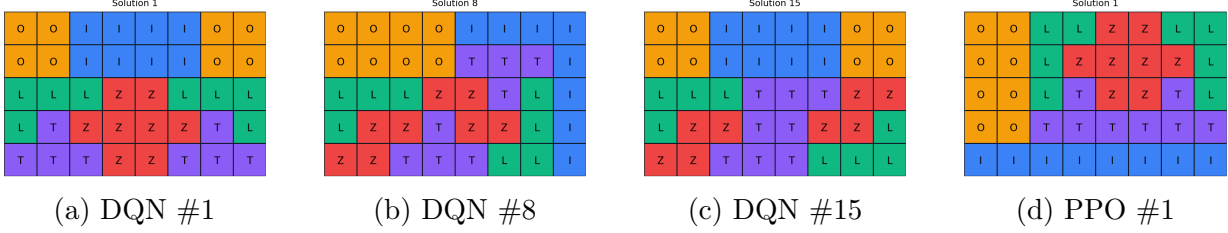


Figure 4: Standard mode solutions (seed 0). DQN discovers 25 structurally different configurations while PPO converges to a single layout.

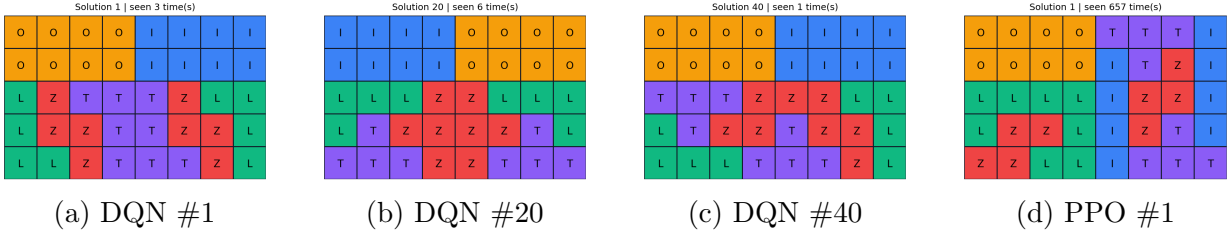


Figure 5: Diverse mode solutions (DQN seed 0, PPO seed 42). DQN finds 46 unique solutions during evaluation; PPO finds at most 3.

9 Failure Analysis

9.1 DQN Diverse Seed 5

The only DQN failure occurs in diverse mode (seed 5): despite discovering 115+ unique solutions during training, the model achieves 0% success at evaluation. This indicates the agent found solutions through ε -greedy exploration but never internalized them into a reliable greedy policy. The diversity penalty may have destabilized value estimates by constantly changing the reward landscape as new solutions were discovered.

9.2 PPO Standard Seed 4

PPO’s single failure in standard mode (seed 4, 0% success) is a classic sparse-reward starvation scenario. PPO is on-policy, so it must discover a complete solution through its own stochastic policy before it can learn to reproduce it. If the random queue orderings in the critical early episodes never produce a solvable trajectory, the policy stagnates.

9.3 PPO Mode Collapse in Diverse Mode

PPO achieves near-perfect success in diverse mode (96–100%) but finds only 1–3 unique solutions. During training, PPO discovers 5–13 solutions but quickly converges to a single dominant one, repeating it over 100,000 times. The entropy coefficient (0.02) is too weak to counteract this collapse, and the diversity penalty (max 4.0) still leaves a positive reward

(1.1) for repeats, so PPO learns that repeating a known solution is safer than risking a dead-end while exploring.

10 Conclusions and Future Work

10.1 Conclusions

1. Both DQN and PPO can solve the BrainBlock puzzle when given sufficient training time (200,000 episodes).
2. **Standard DQN is the most reliable solver:** 100% success on all 7 seeds, with up to 25 unique solutions at evaluation.
3. **Diverse DQN maximizes solution diversity:** up to 46 unique solutions at evaluation and 170+ during training, thanks to the diversity penalty and ϵ -greedy exploration.
4. PPO converges faster but suffers from **mode collapse**, finding only 1–3 solutions regardless of the reward scheme.
5. Action masking is essential for tractable learning in this combinatorial action space.
6. **Training budget is critical:** at 50,000 episodes, DQN fails completely; at 200,000, it achieves 100% success across all seeds.

10.2 Future Improvements

- **Convolutional backbone:** process the board as a 2D grid to learn spatial patterns more efficiently.
- **Dense reward shaping:** add intermediate signals based on remaining valid placements or board compactness.
- **Higher entropy for PPO:** increase the entropy coefficient to counteract mode collapse in diverse mode.
- **GAE:** replace Monte Carlo returns with Generalized Advantage Estimation for lower-variance PPO updates.

11 AI Usage

In accordance with academic integrity guidelines, we declare the usage of the following external resources and generative AI tools in this project:

- **Generative AI Assistants:** The Antigravity AI coding assistant (powered by Gemini) was utilized during the development phase to assist in refactoring Python scripts, optimizing state representations, and automating the multi-seed evaluation pipeline.
- **Report Drafting:** Generative AI tools (specifically Claude Opus via the Antigravity interface) were used to assist in drafting, proofreading, and refining sections of this final report.

All final code modifications, training experiments, results analysis, and text synthesis were thoroughly reviewed, verified, and compiled by the authors.